
Breast Cancer Wisconsin

Relazione di Consegna

Stefano Valloncini, Matricola 0322500114

Indice

1	Introduzione	1
2	Dataset e preprocessing	1
3	Analisi delle feature con PCA	2
4	Algoritmi di classificazione	3
4.1	Regressione logistica	4
4.2	Random forest	4
4.3	Support Vector Machine con kernel RBF	4
4.4	K-Nearest Neighbors	4
5	Metriche di valutazione	4
6	Matrice di confusione e curve ROC	6
7	Approccio di Deep Learning: MLP	6
8	Conclusioni	8
9	Repository GitHub	9
10	Appendice: codice completo del notebook	9

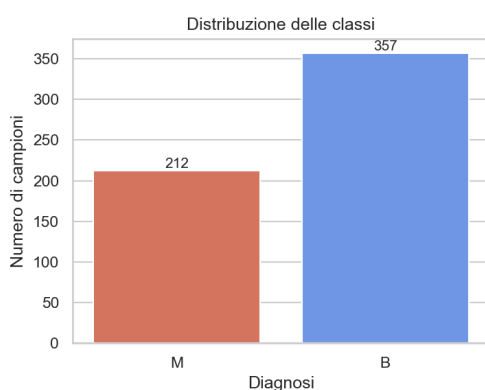
1 Introduzione

Il progetto consiste nello sviluppo di un esperimento di classificazione binaria sul dataset *Breast Cancer Wisconsin (Diagnostic)*, disponibile nel repository UCI con identificativo 17. L'obiettivo è discriminare le masse mammarie maligne (M) dalle benigne (B) a partire da trenta misure morfologiche estratte da immagini di agoaspirato.

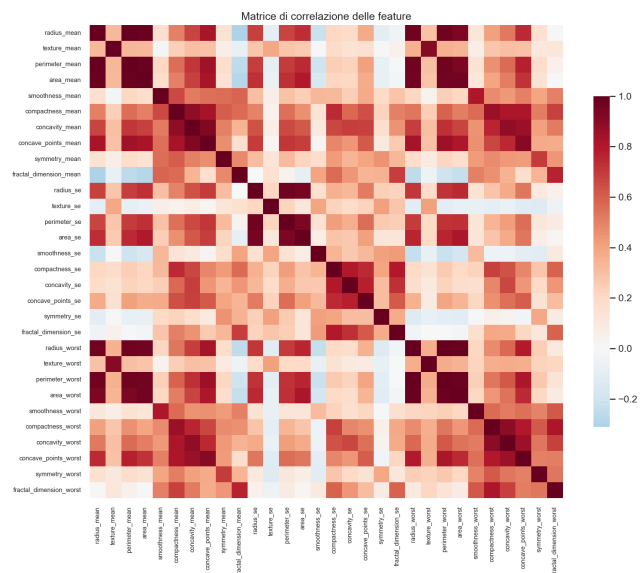
La consegna richiede cinque task: analisi delle feature tramite PCA, implementazione di almeno tre algoritmi di classificazione, calcolo di *precision*, *recall*, F-measure, *accuracy* e ROC AUC, visualizzazione di matrice di confusione e curva ROC, e infine una scelta fra un approccio di deep learning sugli stessi dati e un approccio di spiegabilità. Ho optato per il deep learning, implementando un perceptrone multilivello (MLP) addestrato sulla stessa partizione training/validation/test usata per gli algoritmi shallow.

2 Dataset e preprocessing

Il dataset Wisconsin Diagnostic [2], raccolto a metà degli anni Novanta presso l'Università del Wisconsin da Wolberg, Street e Mangasarian, contiene 569 osservazioni e 30 feature numeriche. Le feature sono organizzate in tre blocchi di dieci: media, errore standard e «worst» (la media dei tre valori peggiori) delle dieci grandezze morfologiche calcolate sui nuclei cellulari (raggio, texture, perimetro, area, smoothness, compattezza, concavità, punti concavi, simmetria, dimensione frattale). La distribuzione delle classi è moderatamente sbilanciata (Fig. 1a): 357 casi benigni (62.7%) e 212 maligni (37.3%). Non sono presenti valori mancanti.



(a) Distribuzione delle classi.



(b) Correlazioni fra le 30 feature.

Figura 1: Sguardo iniziale ai dati: bilanciamento delle classi e correlazioni fra le feature.

La heatmap di correlazione in Fig. 1b conferma l'attesa: i tre blocchi *mean*, *se* e *worst* della stessa grandezza sono fortemente correlati fra loro, e all'interno del blocco *mean* le quantità geometriche (raggio, perimetro, area) hanno correlazioni vicine a 1. È un punto da tenere a mente per la PCA della sezione successiva.

Ho suddiviso il dataset in training, validation e test con strategia stratificata, così la proporzione fra classi resta costante nei tre sottoinsiemi. Il validation set mi serve esplicitamente per la early-stopping della rete neurale; il tuning degli algoritmi shallow passa invece per cross-validation interna al training set.

Sottoinsieme	Campioni	Casi maligni
Training set	341	127
Validation set	114	43
Test set	114	42

Le proporzioni 60/20/20 sono ottenute con due chiamate successive a `train_test_split`, fissando `random_state=42` per la riproducibilità.

Sul training set è stato fittato uno `StandardScaler` di scikit-learn [3]; la trasformazione viene poi applicata identicamente a validation e test. Lo scaling è indispensabile per KNN, SVM e la rete MLP, che dipendono da distanze nello spazio delle feature, ed è innocuo per gli algoritmi basati su alberi (*tree-based*); per uniformità ho applicato la stessa pipeline a tutti i modelli.

3 Analisi delle feature con PCA

L'analisi delle componenti principali (PCA) è stata calcolata sul training set standardizzato. La Fig. 2 mostra l'andamento della varianza spiegata cumulata: bastano 10 componenti per coprire oltre il 95% della varianza totale e già 6 componenti superano l'85%. Coerentemente con quanto si vede in Fig. 1b, la collinearità fra le 30 feature originali permette di comprimere lo spazio senza perdere informazione significativa.

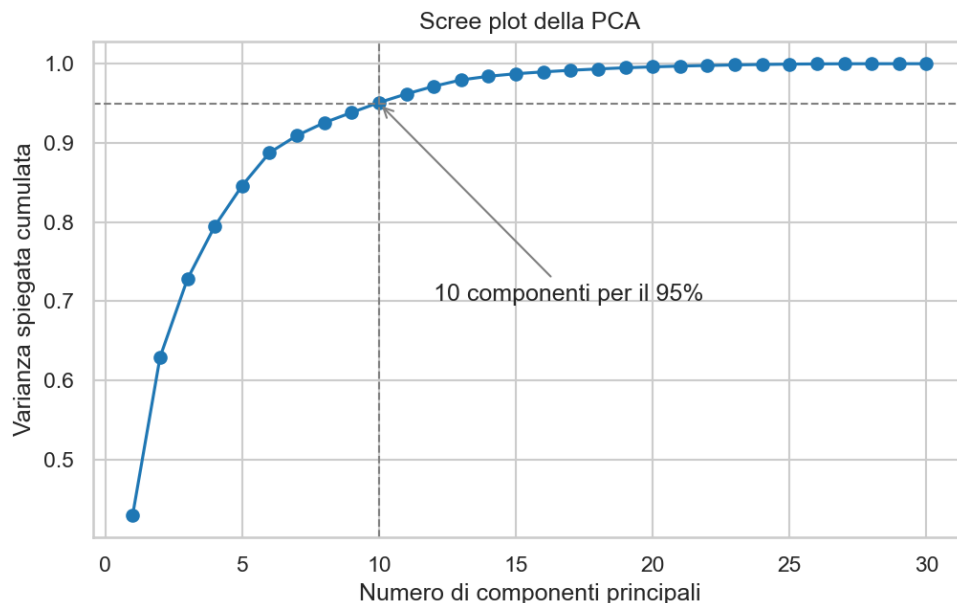


Figura 2: Scree plot della PCA sul training set standardizzato. La linea tratteggiata indica la soglia del 95% di varianza cumulata, raggiunta a 10 componenti.

La proiezione bidimensionale sulle prime due componenti principali (Fig. 3) chiarisce visivamente il motivo per cui anche modelli relativamente semplici ottengono ottime prestazioni: nello spazio delle

prime due PC le due classi sono ampiamente separabili, con una sovrapposizione limitata a una zona di transizione. La prima componente da sola cattura il 43% della varianza ed è correlata con le dimensioni del nucleo, mentre la seconda (circa il 20%) raccoglie la variabilità di concavità e simmetria.

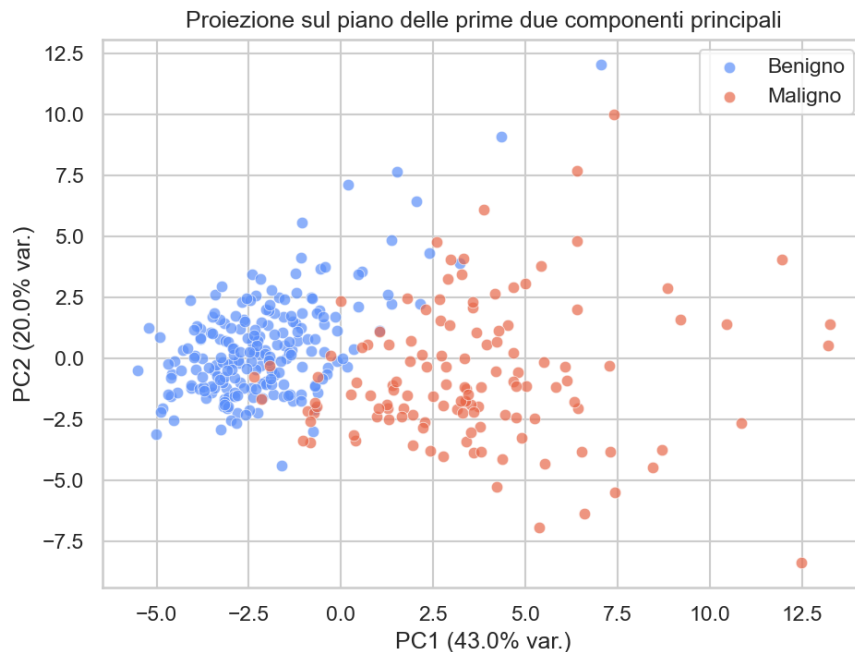


Figura 3: Proiezione del training set sulle prime due componenti principali. I casi maligni occupano la regione con valori più alti della PC1.

Per i modelli finali non ho applicato la PCA come riduzione: con 30 feature il costo computazionale è già basso, e tenere lo spazio originale lascia leggibili le variabili una per una. Lo scree plot e il piano delle prime due componenti mi sono serviti soprattutto come check iniziale per capire che il problema, una volta standardizzato, era già abbastanza separabile.

4 Algoritmi di classificazione

Ho implementato quattro algoritmi shallow fra quelli affrontati nel corso: regressione logistica, random forest, SVM con kernel RBF e k -NN. Per ogni modello ho fatto una piccola grid search con 5-fold cross-validation interna al training set, scegliendo gli iperparametri che massimizzano l'AUC. Il modello finale è poi stato rifittato su training+validation e valutato una sola volta sul test set; un esempio di pattern di codice usato è il seguente.

```
grid_svm = GridSearchCV(
    SVC(kernel="rbf", probability=True, random_state=SEED),
    param_grid={"C": [0.1, 1, 10], "gamma": ["scale", 0.01, 0.1]},
    cv=5, scoring="roc_auc", n_jobs=-1,
)
grid_svm.fit(X_train_s, y_train)
best_svm = grid_svm.best_estimator_.fit(X_trainval_s, y_trainval)
```

4.1 Regressione logistica

La regressione logistica è il modello lineare di riferimento per la classificazione binaria: stima la probabilità della classe positiva come $\sigma(w^\top x + b)$, dove σ è la sigmoide. Con feature standardizzate funziona molto bene su problemi quasi-linearmente separabili. La griglia $C \in \{0.01, 0.1, 1, 10\}$ controlla la regolarizzazione L2: il valore ottimale risultato è $C = 10$, ovvero la regolarizzazione più debole della griglia. Va letto come segnale che con i dati a disposizione il modello non soffre di overfitting e può sfruttare a pieno i pesi.

4.2 Random forest

Un random forest è un ensemble di alberi decisionali addestrati su bootstrap sample del training set con scelta casuale delle feature ad ogni split; la predizione finale è il voto di maggioranza. È robusto al rumore, non richiede scaling delle feature e fornisce naturalmente una stima di importanza delle variabili. La griglia esplorata è $n_estimators \in \{100, 200\}$, $max_depth \in \{None, 5, 10\}$; la configurazione ottimale è $n_estimators = 100$ e $max_depth = 5$, indizio che un controllo della complessità degli alberi giova alla generalizzazione su un dataset di poche centinaia di campioni.

4.3 Support Vector Machine con kernel RBF

L'SVM con kernel gaussiano cerca l'iperpiano a margine massimo in uno spazio delle feature implicitamente espanso. Il flag `probability=True` attiva la calibrazione di Platt per ottenere stime di probabilità, indispensabili per il calcolo della ROC AUC. Sulla griglia $C \in \{0.1, 1, 10\}$, $\gamma \in \{scale, 0.01, 0.1\}$, la combinazione vincente è $C = 10$ e $\gamma = 0.01$: parametri che producono un modello flessibile ma non eccessivamente *wiggly*.

4.4 K-Nearest Neighbors

Il k -NN classifica un nuovo punto in base alla classe più frequente fra i suoi k vicini nello spazio standardizzato. Sulla griglia $k \in \{3, 5, 7, 11\}$ con pesi *uniform* o *distance*, la configurazione ottimale è $k = 11$ con pesi proporzionali all'inverso della distanza. Un k relativamente grande tende a smussare il decision boundary; abbinato ai pesi per distanza, fa da regolarizzazione contro il rumore.

5 Metriche di valutazione

Le metriche sono quelle richieste dalla consegna e definite a lezione. Dati TP, TN, FP e FN (veri/falsi positivi e negativi):

$$\text{Precision} = \frac{TP}{TP + FP}, \quad \text{Recall} = \frac{TP}{TP + FN}, \quad F_1 = \frac{2 \cdot \text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$$

con accuracy pari alla frazione di predizioni corrette sul totale. La ROC AUC è invece l'area sottesa dalla curva che mette in relazione *True Positive Rate* (= recall) e *False Positive Rate* al variare della soglia di decisione; come riportato in [1], è la misura più robusta quando le classi non sono perfettamente bilanciate, perché non dipende dalla scelta della soglia.

Per il calcolo ho usato direttamente le funzioni di `sklearn.metrics`, racchiuse in una piccola funzione che restituisce un dizionario:

```
def evaluate(model, X_te, y_te, name):
```

```

if hasattr(model, "predict_proba"):
    proba = model.predict_proba(X_te)[: , 1]
else:
    proba = model.decision_function(X_te)
pred = (proba >= 0.5).astype(int)
return {
    "accuracy": accuracy_score(y_te, pred),
    "precision": precision_score(y_te, pred),
    "recall": recall_score(y_te, pred),
    "f1": f1_score(y_te, pred),
    "roc_auc": roc_auc_score(y_te, proba),
}

```

La [Tabella 1](#) riassume i risultati di tutti e cinque i modelli sul test set; la [Fig. 4](#) ne dà la rappresentazione visiva.

Modello	Accuracy	Precision	Recall	F1	ROC AUC
Logistic Regression	0.9737	0.9756	0.9524	0.9639	0.9848
Random Forest	0.9737	0.9756	0.9524	0.9639	0.9964
SVM (RBF)	0.9825	1.0000	0.9524	0.9756	0.9947
K-Nearest Neighbors	0.9737	0.9756	0.9524	0.9639	0.9934
MLP	0.9825	1.0000	0.9524	0.9756	0.9947

Tabella 1: Metriche di valutazione sul test set (114 campioni, 42 maligni). In grassetto il valore migliore per ciascuna colonna.

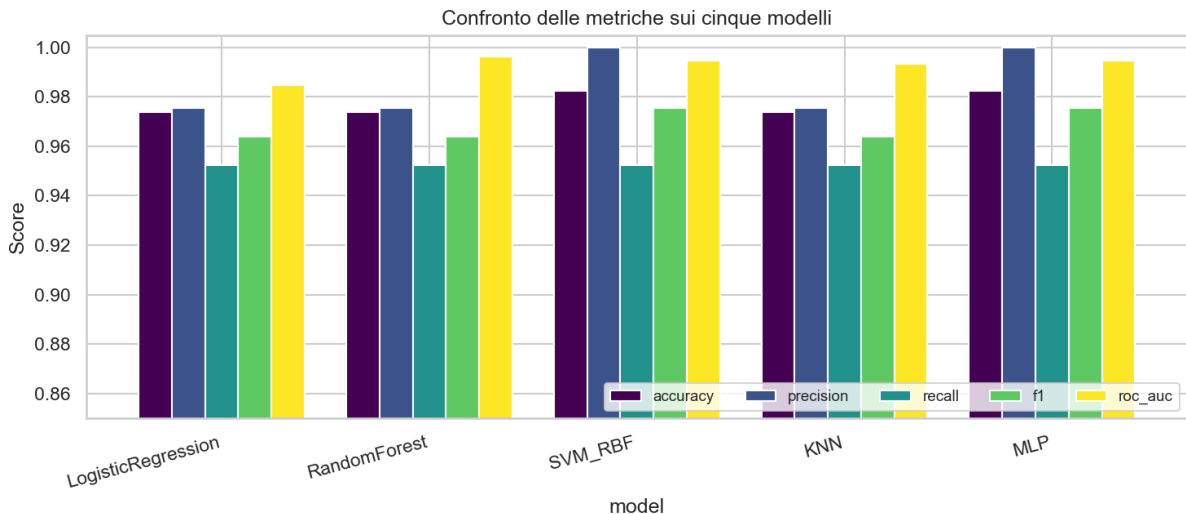


Figura 4: Confronto visivo delle cinque metriche per i cinque modelli. L'asse delle ordinate è zoomato sopra 0.85 per evidenziare le differenze.

SVM con kernel RBF e MLP sono testa a testa sulla maggior parte delle metriche: stessa accuracy (0.9825), stessa F_1 (0.9756) e precision pari a 1 per entrambi, cioè nessun falso positivo. Random Forest porta a casa l'AUC più alta, 0.9964, praticamente al massimo. Detto questo, le differenze fra i modelli sono piccole e tutti mantengono recall a 0.9524.

6 Matrice di confusione e curve ROC

La Fig. 5 mostra le cinque matrici di confusione sul test set. Tutti i modelli sbagliano fra le 2 e le 3 predizioni su 114: l'errore più frequente è il falso negativo, cioè un caso maligno classificato come benigno, clinicamente l'errore più grave.

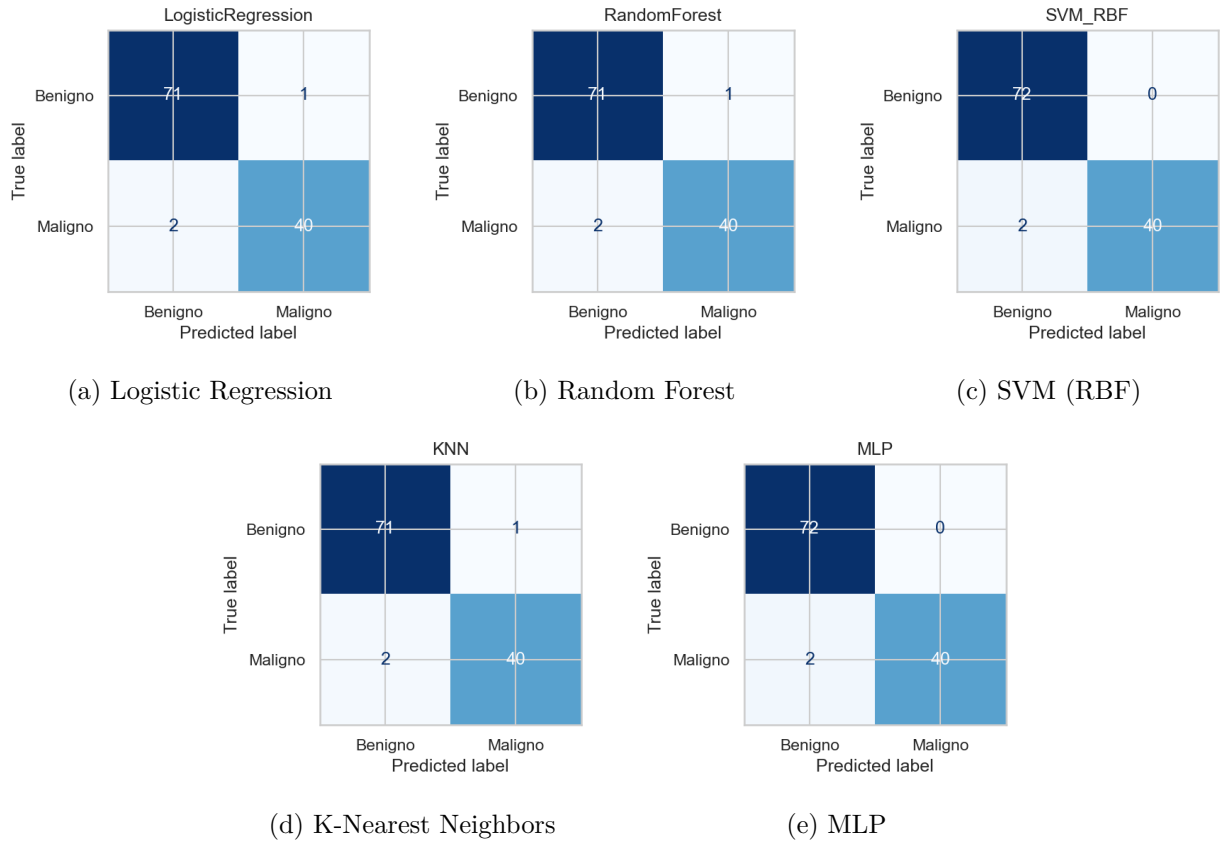


Figura 5: Matrici di confusione dei cinque modelli sul test set.

La Fig. 6 sovrappone le cinque curve ROC. Le curve sono tutte schiacciate nell'angolo in alto a sinistra, segno di classificatori molto discriminanti; le differenze di AUC fra i modelli sono nell'ordine del centesimo.

7 Approccio di Deep Learning: MLP

Il modello deep, implementato in Keras su TensorFlow [4], è un percettrone multilivello (*multi-layer perceptron*) con due hidden layer e regolarizzazione tramite dropout. La scelta è volutamente sobria: dato il numero ridotto di campioni (poche centinaia), architetture più profonde o più larghe rischierebbero un forte overfitting senza beneficio in generalizzazione.

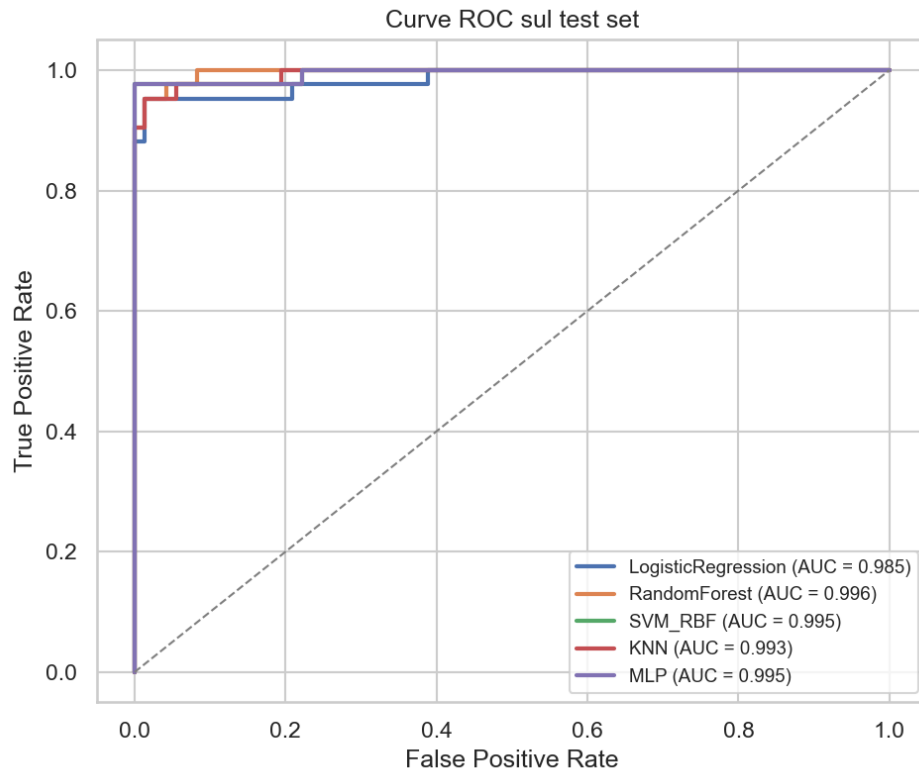


Figura 6: Curve ROC sul test set. La diagonale tratteggiata corrisponde al classificatore casuale (AUC = 0.5).

Layer	Configurazione	# Parametri
Input	30 unit	0
Dense	64 unit, attivazione ReLU	1 984
Dropout	rate 0.3	0
Dense	32 unit, attivazione ReLU	2 080
Dropout	rate 0.2	0
Dense (output)	1 unit, sigmoide	33
Totale		4 097

La loss è *binary_crossentropy*, l'ottimizzatore Adam con learning rate 10^{-3} . Il training usa *early stopping* sul validation loss con `patience=10` e ripristino dei pesi migliori. Per avere risultati riproducibili ho abilitato il determinismo di TensorFlow (`enable_op_determinism`) e fissato il seed globale di Keras. Con questo setup la rete si ferma stabilmente dopo 38 epoche.

```
def build_mlp(input_dim):
    model = keras.Sequential([
        layers.Input(shape=(input_dim,)),
        layers.Dense(64, activation="relu"),
        layers.Dropout(0.3),
        layers.Dense(32, activation="relu"),
        layers.Dropout(0.2),
        layers.Dense(1, activation="sigmoid"),
    ])
    model.compile(
        optimizer=keras.optimizers.Adam(learning_rate=1e-3),
```

```

    loss="binary_crossentropy",
    metrics=["accuracy", keras.metrics.AUC(name="auc")],
)
return model

mlp = build_mlp(X_train_s.shape[1])
early = callbacks.EarlyStopping(
    monitor="val_loss", patience=10, restore_best_weights=True,
)
history = mlp.fit(
    X_train_s, y_train.values,
    validation_data=(X_val_s, y_val.values),
    epochs=100, batch_size=32,
    callbacks=[early], verbose=0,
)

```

Le curve di training nella Fig. 7 mostrano una convergenza rapida: la loss di validazione scende stabilmente nelle prime epoche per poi appiattirsi, segno che l'*early stopping* ha colto bene il punto di arresto. Sul test set la rete ottiene esattamente le stesse metriche dell'SVM con kernel RBF (accuracy 0.9825, F_1 0.9756, AUC 0.9947), con due falsi negativi e zero falsi positivi.

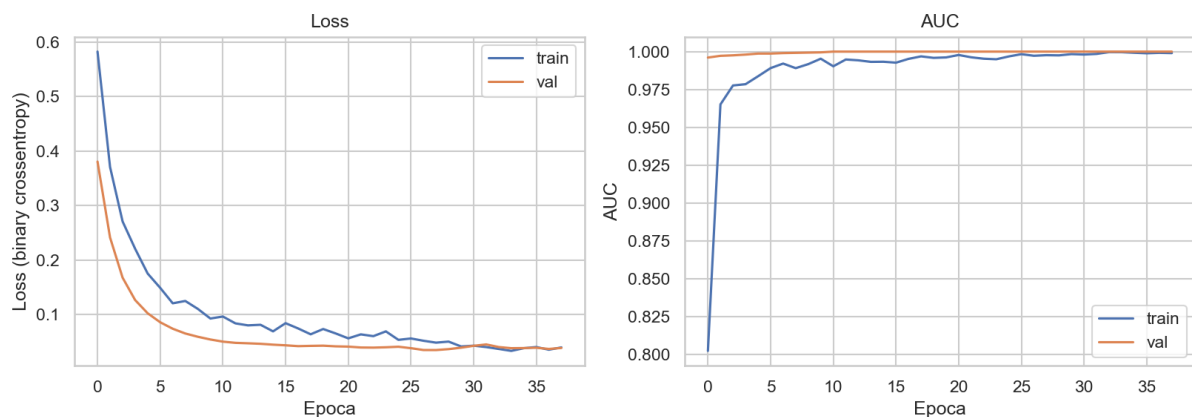


Figura 7: Curve di loss e AUC su training e validation durante il fitting del MLP.

8 Conclusioni

Sul dataset Breast Cancer Wisconsin il problema di classificazione si è rivelato relativamente *facile*: già la regressione logistica raggiunge un'accuracy del 97.4% e un AUC del 98.5%, e i modelli più espressivi chiudono il margine residuo. SVM con kernel RBF e MLP arrivano allo stesso punto operativo (accuracy 98.25%, F_1 0.9756), mentre Random Forest tiene l'AUC più alta, 0.9964.

Sul piano pratico la scelta fra SVM e MLP è quasi indifferente: a parità di metriche tenderei a preferire SVM, perché è più semplice da spiegare, più veloce da addestrare e non richiede l'apparato di setup per il determinismo che la MLP impone se si vuole un risultato riproducibile. In un contesto clinico, dove il falso negativo è l'errore più grave, abbasserei la soglia di decisione sotto 0.5 per spostare il modello verso una recall più alta a costo di qualche falso allarme: la curva ROC permette di scegliere il punto di lavoro senza dover riaddestrare nulla.

Restano alcuni limiti. La stima dell'errore di generalizzazione poggia su un unico test set di 114

campioni: una nested cross-validation darebbe stime più robuste e separerebbe la selezione degli iperparametri dalla valutazione finale. Manca un test di significatività statistica fra i modelli: con accuracy così vicine basta poco a ribaltare la classifica. Un'estensione naturale sarebbe SHAP, per capire quali fra le trenta feature pesano davvero nelle predizioni del modello migliore.

9 Repository GitHub

Il notebook e tutto il materiale del progetto sono pubblicati nel repository di consegna del corso:

https://github.com/NScarpato79/apprendimento_automatico_e_apprendimento_profondo

10 Appendice: codice completo del notebook

Si riporta il codice integrale di `notebook.ipynb`, nell'ordine di esecuzione delle celle.

```
import os
os.environ["PYTHONHASHSEED"] = "42"
os.environ["TF_DETERMINISTIC_OPS"] = "1"

import warnings
warnings.filterwarnings("ignore")

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from pathlib import Path

from ucimlrepo import fetch_ucirepo
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
from sklearn.svm import SVC
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import (
    accuracy_score, precision_score, recall_score, f1_score, roc_auc_score,
    confusion_matrix, ConfusionMatrixDisplay, roc_curve,
)

import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers, callbacks

SEED = 42
np.random.seed(SEED)
keras.utils.set_random_seed(SEED)
tf.config.experimental.enable_op_determinism()

IMG_DIR = Path("docs/images")
IMG_DIR.mkdir(parents=True, exist_ok=True)
```

```
sns.set_theme(style="whitegrid", context="notebook")
```

Caricamento del dataset

```
data = fetch_ucirepo(id=17)
X = data.data.features.copy()
y_raw = data.data.targets.squeeze()

# Le colonne UCI sono radius1/2/3, texture1/2/3, ecc.: il suffisso 1/2/3
# corrisponde a mean/se/worst. Le rinomino per leggerle senza fatica nei plot.
SUFFIX = {"1": "mean", "2": "se", "3": "worst"}
X.columns = [f"{c[:-1]}_{SUFFIX[c[-1]]}" for c in X.columns]

print(f"Numero di campioni: {len(X)}")
print(f"Numero di feature : {X.shape[1]}")
print("Distribuzione delle classi:")
print(y_raw.value_counts())
```

```
X.head()
```

Analisi esplorativa

```
y = (y_raw == "M").astype(int)

fig, ax = plt.subplots(figsize=(5, 4))
sns.countplot(x=y_raw, ax=ax, palette={"B": "#5B8FF9", "M": "#E8684A"})
ax.set_xlabel("Diagnosi")
ax.set_ylabel("Numero di campioni")
ax.set_title("Distribuzione delle classi")
for p in ax.patches:
    ax.annotate(int(p.get_height()), (p.get_x() + p.get_width() / 2, p.get_height()),
                ha="center", va="bottom", fontsize=10)
plt.tight_layout()
plt.savefig(IMG_DIR / "class_distribution.png", dpi=150)
plt.show()
```

```
# Le 30 feature sono organizzate in tre blocchi (mean, error, worst): le
# correlazioni piu' forti si trovano fra blocchi diversi sulla stessa quantita'
# (radius_mean vs radius_worst, ecc.).
fig, ax = plt.subplots(figsize=(11, 9))
corr = X.corr()
sns.heatmap(corr, cmap="RdBu_r", center=0, square=True,
            cbar_kws={"shrink": 0.7}, ax=ax, xticklabels=True, yticklabels=True)
ax.set_title("Matrice di correlazione delle feature")
plt.xticks(rotation=90, fontsize=7)
plt.yticks(fontsize=7)
plt.tight_layout()
plt.savefig(IMG_DIR / "correlation_heatmap.png", dpi=150)
plt.show()

print("Valori mancanti per colonna:", X.isnull().sum().sum())
```

Suddivisione dei dati e standardizzazione

```
X_train, X_tmp, y_train, y_tmp = train_test_split(
    X, y, test_size=0.4, stratify=y, random_state=SEED
)
X_val, X_test, y_val, y_test = train_test_split(
    X_tmp, y_tmp, test_size=0.5, stratify=y_tmp, random_state=SEED
)

scaler = StandardScaler().fit(X_train)
X_train_s = scaler.transform(X_train)
X_val_s = scaler.transform(X_val)
X_test_s = scaler.transform(X_test)

print(f"Train: {X_train_s.shape}, positivi: {y_train.sum()}/{len(y_train)}")
print(f"Val : {X_val_s.shape}, positivi: {y_val.sum()}/{len(y_val)}")
print(f"Test : {X_test_s.shape}, positivi: {y_test.sum()}/{len(y_test)}")
```

Analisi PCA

```
pca_full = PCA(random_state=SEED).fit(X_train_s)
cum_var = np.cumsum(pca_full.explained_variance_ratio_)
k95 = int(np.searchsorted(cum_var, 0.95)) + 1

fig, ax = plt.subplots(figsize=(7, 4.5))
ax.plot(range(1, len(cum_var) + 1), cum_var, marker="o", color="#1f77b4")
ax.axhline(0.95, ls="--", color="gray", lw=1)
ax.axvline(k95, ls="--", color="gray", lw=1)
ax.set_xlabel("Numero di componenti principali")
ax.set_ylabel("Varianza spiegata cumulata")
ax.set_title("Scree plot della PCA")
ax.annotate(f"{k95} componenti per il 95%", xy=(k95, 0.95),
            xytext=(k95 + 2, 0.7),
            arrowprops=dict(arrowstyle="->", color="gray"))
plt.tight_layout()
plt.savefig(IMG_DIR / "pca_variance.png", dpi=150)
plt.show()

print(f"Componenti necessarie per >=95% varianza: {k95}")
```

```
pca2 = PCA(n_components=2, random_state=SEED).fit(X_train_s)
Z_train = pca2.transform(X_train_s)

fig, ax = plt.subplots(figsize=(6.5, 5))
for label, color, name in [(0, "#5B8FF9", "Benigno"), (1, "#E8684A", "Maligno")]:
    mask = y_train.values == label
    ax.scatter(Z_train[mask, 0], Z_train[mask, 1], c=color, label=name,
               alpha=0.7, edgecolor="white", linewidth=0.4)
ax.set_xlabel(f"PC1 ({pca2.explained_variance_ratio_[0]*100:.1f}% var.)")
ax.set_ylabel(f"PC2 ({pca2.explained_variance_ratio_[1]*100:.1f}% var.)")
ax.set_title("Proiezione sul piano delle prime due componenti principali")
ax.legend()
plt.tight_layout()
plt.savefig(IMG_DIR / "pca_2d_scatter.png", dpi=150)
plt.show()
```

Algoritmi shallow

```
def evaluate(model, X_te, y_te, name):
    if hasattr(model, "predict_proba"):
        proba = model.predict_proba(X_te)[:, 1]
    else:
        proba = model.decision_function(X_te)
    pred = (proba >= 0.5).astype(int)
    return {
        "model": name,
        "accuracy": accuracy_score(y_te, pred),
        "precision": precision_score(y_te, pred),
        "recall": recall_score(y_te, pred),
        "f1": f1_score(y_te, pred),
        "roc_auc": roc_auc_score(y_te, proba),
        "proba": proba,
        "pred": pred,
    }

X_trainval_s = np.vstack([X_train_s, X_val_s])
y_trainval = pd.concat([y_train, y_val]).reset_index(drop=True)

results = {}
grids = {}
```

```
grid_lr = GridSearchCV(
    LogisticRegression(max_iter=5000, solver="liblinear", random_state=SEED),
    param_grid={"C": [0.01, 0.1, 1, 10], "penalty": ["l2"]},
    cv=5, scoring="roc_auc", n_jobs=-1,
)
grid_lr.fit(X_train_s, y_train)
best_lr = grid_lr.best_estimator_.fit(X_trainval_s, y_trainval)
grids["LogisticRegression"] = grid_lr.best_params_
results["LogisticRegression"] = evaluate(best_lr, X_test_s, y_test, "LogisticRegression")
print("Best params:", grid_lr.best_params_)
```

```
grid_rf = GridSearchCV(
    RandomForestClassifier(random_state=SEED, n_jobs=-1),
    param_grid={"n_estimators": [100, 200], "max_depth": [None, 5, 10]},
    cv=5, scoring="roc_auc", n_jobs=-1,
)
grid_rf.fit(X_train_s, y_train)
best_rf = grid_rf.best_estimator_.fit(X_trainval_s, y_trainval)
grids["RandomForest"] = grid_rf.best_params_
results["RandomForest"] = evaluate(best_rf, X_test_s, y_test, "RandomForest")
print("Best params:", grid_rf.best_params_)
```

```
# probability=True abilita predict_proba via Platt scaling, indispensabile per ROC AUC.
grid_svm = GridSearchCV(
    SVC(kernel="rbf", probability=True, random_state=SEED),
    param_grid={"C": [0.1, 1, 10], "gamma": ["scale", 0.01, 0.1]},
    cv=5, scoring="roc_auc", n_jobs=-1,
)
grid_svm.fit(X_train_s, y_train)
best_svm = grid_svm.best_estimator_.fit(X_trainval_s, y_trainval)
grids["SVM_RBF"] = grid_svm.best_params_
```

```
results["SVM_RBF"] = evaluate(best_svm, X_test_s, y_test, "SVM_RBF")
print("Best params:", grid_svm.best_params_)
```

```
grid_knn = GridSearchCV(
    KNeighborsClassifier(),
    param_grid={"n_neighbors": [3, 5, 7, 11], "weights": ["uniform", "distance"]},
    cv=5, scoring="roc_auc", n_jobs=-1,
)
grid_knn.fit(X_train_s, y_train)
best_knn = grid_knn.best_estimator_.fit(X_trainval_s, y_trainval)
grids["KNN"] = grid_knn.best_params_
results["KNN"] = evaluate(best_knn, X_test_s, y_test, "KNN")
print("Best params:", grid_knn.best_params_)
```

Rete neurale (MLP)

```
def build_mlp(input_dim):
    model = keras.Sequential([
        layers.Input(shape=(input_dim,)),
        layers.Dense(64, activation="relu"),
        layers.Dropout(0.3),
        layers.Dense(32, activation="relu"),
        layers.Dropout(0.2),
        layers.Dense(1, activation="sigmoid"),
    ])
    model.compile(
        optimizer=keras.optimizers.Adam(learning_rate=1e-3),
        loss="binary_crossentropy",
        metrics=["accuracy", keras.metrics.AUC(name="auc")],
    )
    return model

mlp = build_mlp(X_train_s.shape[1])
mlp.summary()
```

```
early = callbacks.EarlyStopping(
    monitor="val_loss", patience=10, restore_best_weights=True
)

history = mlp.fit(
    X_train_s, y_train.values,
    validation_data=(X_val_s, y_val.values),
    epochs=100, batch_size=32,
    callbacks=[early], verbose=0,
)

print(f"Training fermato dopo {len(history.history['loss'])} epoche")
```

```
fig, axes = plt.subplots(1, 2, figsize=(11, 4))

axes[0].plot(history.history["loss"], label="train")
axes[0].plot(history.history["val_loss"], label="val")
axes[0].set_xlabel("Epoca")
axes[0].set_ylabel("Loss (binary crossentropy)")
axes[0].set_title("Loss")
```

```

axes[0].legend()

axes[1].plot(history.history["auc"], label="train")
axes[1].plot(history.history["val_auc"], label="val")
axes[1].set_xlabel("Epoca")
axes[1].set_ylabel("AUC")
axes[1].set_title("AUC")
axes[1].legend()

plt.tight_layout()
plt.savefig(IMG_DIR / "mlp_training_history.png", dpi=150)
plt.show()

```

```

class KerasProbaWrap:
    """Adattatore minimo per riutilizzare la funzione evaluate() sulla MLP."""
    def __init__(self, model):
        self.model = model
    def predict_proba(self, X):
        p = self.model.predict(X, verbose=0).ravel()
        return np.column_stack([1 - p, p])

results["MLP"] = evaluate(KerasProbaWrap(mlp), X_test_s, y_test, "MLP")

```

Metriche di valutazione

```

metrics_df = pd.DataFrame(
    [{k: v for k, v in r.items() if k in ("model", "accuracy", "precision", "recall", "f1",
    ↪ "roc_auc")}]
    for r in results.values())
).set_index("model").round(4)

metrics_df

```

```

fig, ax = plt.subplots(figsize=(10, 4.5))
metrics_df.plot(kind="bar", ax=ax, colormap="viridis", width=0.8)
ax.set_ylim(0.85, 1.005)
ax.set_ylabel("Score")
ax.set_title("Confronto delle metriche sui cinque modelli")
ax.legend(loc="lower right", ncol=5, fontsize=9)
plt.xticks(rotation=15, ha="right")
plt.tight_layout()
plt.savefig(IMG_DIR / "metrics_comparison.png", dpi=150)
plt.show()

```

Matrici di confusione

```

name_to_file = {
    "LogisticRegression": "confusion_matrix_logreg.png",
    "RandomForest":      "confusion_matrix_rf.png",
    "SVM_RBF":           "confusion_matrix_svm.png",
    "KNN":               "confusion_matrix_knn.png",
    "MLP":               "confusion_matrix_mlp.png",
}

```

```

fig, axes = plt.subplots(2, 3, figsize=(12, 7.5))
axes = axes.ravel()

for ax, (name, r) in zip(axes, results.items()):
    cm = confusion_matrix(y_test, r["pred"])
    disp = ConfusionMatrixDisplay(cm, display_labels=["Benigno", "Maligno"])
    disp.plot(ax=ax, cmap="Blues", colorbar=False)
    ax.set_title(name)
    fig.savefig(IMG_DIR / name_to_file[name], dpi=150, bbox_inches="tight")

axes[-1].axis("off")
plt.tight_layout()
plt.show()

# Salvataggio dei singoli plot in file separati per essere inclusi nella relazione.
for name, r in results.items():
    cm = confusion_matrix(y_test, r["pred"])
    fig_s, ax_s = plt.subplots(figsize=(4, 3.5))
    ConfusionMatrixDisplay(cm, display_labels=["Benigno", "Maligno"]).plot(
        ax=ax_s, cmap="Blues", colorbar=False
    )
    ax_s.set_title(name)
    plt.tight_layout()
    fig_s.savefig(IMG_DIR / name_to_file[name], dpi=150)
    plt.close(fig_s)

```

Curve ROC

```

fig, ax = plt.subplots(figsize=(6.5, 5.5))
for name, r in results.items():
    fpr, tpr, _ = roc_curve(y_test, r["proba"])
    ax.plot(fpr, tpr, lw=2, label=f"{name} (AUC = {r['roc_auc']:.3f})")
ax.plot([0, 1], [0, 1], ls="--", color="gray", lw=1)
ax.set_xlabel("False Positive Rate")
ax.set_ylabel("True Positive Rate")
ax.set_title("Curve ROC sul test set")
ax.legend(loc="lower right", fontsize=9)
plt.tight_layout()
plt.savefig(IMG_DIR / "roc_curves_all.png", dpi=150)
plt.show()

```

Bibliografia

-
- [1] Gareth James, Daniela Witten, Trevor Hastie, Robert Tibshirani, Jonathan Taylor, *An Introduction to Statistical Learning with Applications in Python*, Springer, 2023. <https://www.statlearning.com/>.
 - [2] W. H. Wolberg, W. N. Street, O. L. Mangasarian, *Breast Cancer Wisconsin (Diagnostic) Data Set*, UCI Machine Learning Repository, 1995. <https://archive.ics.uci.edu/dataset/17/>.
-

-
- [3] F. Pedregosa et al., *Scikit-learn: Machine Learning in Python*, Journal of Machine Learning Research, 12, 2825–2830, 2011.
- [4] M. Abadi et al., *TensorFlow: A System for Large-Scale Machine Learning*, OSDI’16, 265–283, 2016.